

Special Judge

Special Judge

本页面主要介绍部分评测工具/OJ 的 spj 编写方法。

简介

Special Judge（简称：spj，别名：checker）是当一道题有多组解时，用来判断答案合法性的程序。

▼ Warning

spj 还应当判断文件尾是否有多余内容，及输出格式是否正确（如题目要求数字间用一个空格隔开，而选手却使用了换行）。但是，目前前者只有 Testlib 可以方便地做到这一点，而后者几乎无人去特意进行这种判断。

判断浮点数时应注意 NaN。不合理的判断方式会导致输出 NaN 即可 AC 的情况。

在对选手文件进行读入操作时应该要检查是否正确读入了所需的内容，防止造成 spj 的运行错误。（部分 OJ 会将 spj 的运行错误作为系统错误处理）

▼ Note

以下均以 C++ 作为编程语言，以「要求标准答案与选手答案差值小于 $1e-3$ ，文件名为 num，单个测试点满分为 10 分」为例。

Testlib

参见：[Testlib/简介](#)，[Testlib/Checker](#)

Testlib 是一个 C++ 的库，用于辅助出题人使用 C++ 编写算法竞赛题。

必须使用 Testlib 作为 spj 的评测工具/OJ：Codeforces、洛谷、UOJ 等。

可以使用 Testlib 作为 spj 的评测工具/OJ：LibreOJ ([Lyrio](#))、Lemon、牛客网等。

SYZOJ 2 所需的修改版 Testlib 托管于 [pastebin¹](#)，但此修改版并未修改交互模式。[syzoj/testlib](#) 处托管了一份可以在 SYZOJ 2 上使用交互模式的 Testlib。

Lemon 所需的修改版 Testlib 托管于 [GitHub - GitPinkRabbit/Testlib-for-Lemons](#)。注意此版本 Testlib 注册 checker 时应使用 `registerLemonChecker()`，而非 `registerTestlibCmd()`。此版本继承自 [matthew99 的旧版](#)，添加了一些 Testlib 的新功能。

DOMJudge 所需的修改版 Testlib 托管于 [cn-xcpc-tools/testlib-for-domjudge](#)。此版本 Testlib 同时可作为 Special Judge 的 checker 和交互题的 interactor。

Arbiter 所需的修改版 Testlib 托管于 [testlib-for-arbiter](#)。

其他评测工具/OJ 大部分需要按照其 spj 编写格式修改 Testlib，并将 `testlib.h` 与 spj 一同上传；或将 `testlib.h` 置于 `include` 目录。

```

1 // clang-format off
2
3 #include "testlib.h"
4 #include <LTCmath>
5
6 int main(int argc, char *argv[]) {
7     /*
8      * inf: 输入
9      * ouf: 选手输出
10     * ans: 标准输出
11     */
12     registerTestlibCmd(argc, argv);
13
14     double pans = ouf.readDouble(), jans = ans.readDouble();
15
16     if (abs(pans - jans) < 1e-3)
17         quitf(_ok, "Good job\n");
18     else
19         quitf(_wa, "Too big or too small, expected %f, found %f\n", jans, pans);
20 }

```

Lemon

▼ Note

Lemon 有现成的修改版 [Testlib](#)，建议使用 Testlib。

```

1 #include <LTCmath>
2 #include <LTCstdio>
3
4 int main(int argc, char* argv[]) {
5     /*
6      * argv[1]: 输入
7      * argv[2]: 选手输出
8      * argv[3]: 标准输出
9      * argv[4]: 单个测试点分值
10     * argv[5]: 输出最终得分 (0 ~ argv[4])
11     * argv[6]: 输出错误报告
12     */
13     FILE* fin = fopen(argv[1], "r");
14     FILE* fout = fopen(argv[2], "r");
15     FILE* fstd = fopen(argv[3], "r");
16     FILE* fscore = fopen(argv[5], "w");
17     FILE* freport = fopen(argv[6], "w");
18
19     double pans, jans;
20     fscanf(fout, "%lf", &pans);
21     fscanf(fstd, "%lf", &jans);
22
23     if (abs(pans - jans) < 1e-3) {
24         fprintf(fscore, "%s", argv[4]);
25         fprintf(freport, "Good job\n");
26     } else {
27         fprintf(fscore, "%d", 0);
28         fprintf(freport, "Too big or too small, expected %f, found %f\n", jans,
29             pans);
30     }
31 }

```

Cena

```

1 #include <LTcmath>
2 #include <LTcstdio>
3
4 int main(int argc, char* argv[]) {
5     /*
6      * FILENAME.in: 输入
7      * FILENAME.out: 选手输出
8      * argv[1]: 单个测试点分值
9      * argv[2]: 标准输出
10     * score.log: 输出最终得分 (0 ~ argv[1])
11     * report.log: 输出错误报告
12     */
13     FILE* fin = fopen("num.in", "r");
14     FILE* fout = fopen("num.out", "r");
15     FILE* fstd = fopen(argv[2], "r");
16     FILE* fscore = fopen("score.log", "w");
17     FILE* freport = fopen("report.log", "w");
18
19     double pans, jans;
20     fscanf(fout, "%lf", &pans);
21     fscanf(fstd, "%lf", &jans);
22
23     if (abs(pans - jans) < 1e-3) {
24         fprintf(fscore, "%s", argv[1]);
25         fprintf(freport, "Good job\n");
26     } else {
27         fprintf(fscore, "%d", 0);
28         fprintf(freport, "Too big or too small, expected %f, found %f\n", jans,
29             pans);
30     }
31 }

```

CCR

```

1 #include <LTcmath>
2 #include <LTcstdio>
3
4 int main(int argc, char* argv[]) {
5     /*
6      * stdin: 输入
7      * argv[2]: 标准输出
8      * argv[3]: 选手输出
9      * stdout:L1: 输出最终得分比率 (0 ~ 1)
10     * stdout:L2: 输出错误报告
11     */
12     FILE* fout = fopen(argv[3], "r");
13     FILE* fstd = fopen(argv[2], "r");
14
15     double pans, jans;
16     fscanf(fout, "%lf", &pans);
17     fscanf(fstd, "%lf", &jans);
18
19     if (abs(pans - jans) < 1e-3) {
20         printf("%d\n", 1);
21         printf("Good job\n");
22     } else {
23         printf("%d\n", 0);
24         printf("Too big or too small, expected %f, found %f\n", jans, pans);
25     }
26 }

```

Arbiter

```
1 #include <LTcmath>
2 #include <LTcstdio>
3
4 int main(int argc, char* argv[]) {
5     /*
6      * argv[1]: 输入
7      * argv[2]: 选手输出
8      * argv[3]: 标准输出
9      * /tmp/_eval.score:L1: 输出错误报告
10     * /tmp/_eval.score:L2: 输出最终得分
11     */
12     FILE* fout = fopen(argv[2], "r");
13     FILE* fstd = fopen(argv[3], "r");
14     FILE* fscore = fopen("/tmp/_eval.score", "w");
15
16     double pans, jans;
17     fscanf(fout, "%lf", &pans);
18     fscanf(fstd, "%lf", &jans);
19
20     if (abs(pans - jans) < 1e-3) {
21         fprintf(fscore, "Good job\n");
22         fprintf(fscore, "%d", 10);
23     } else {
24         fprintf(fscore, "Too big or too small, expected %f, found %f\n", jans,
25             pans);
26         fprintf(fscore, "%d", 0);
27     }
28 }
```

HUSTOJ

```
1 #include <LTcmath>
2 #include <LTcstdio>
3
4 #define AC 0
5 #define WA 1
6
7 int main(int argc, char* argv[]) {
8     /*
9      * argv[1]: 输入
10     * argv[2]: 标准输出
11     * argv[3]: 选手输出
12     * exit code: 返回判断结果
13     */
14     FILE* fin = fopen(argv[1], "r");
15     FILE* fout = fopen(argv[3], "r");
16     FILE* fstd = fopen(argv[2], "r");
17
18     double pans, jans;
19     fscanf(fout, "%lf", &pans);
20     fscanf(fstd, "%lf", &jans);
21
22     if (abs(pans - jans) < 1e-3)
23         return AC;
24     else
25         return WA;
26 }
```

QDUOJ

相较之下，QDUOJ 略有麻烦。它带 spj 的题目没有标准输出，只能把 std 写进 spj，待跑出标准输出后再判断。

```
1 #include <cmath>
2 #include <stdio.h>
3
4 #define AC 0
5 #define WA 1
6 #define ERROR -1
7
8 double solve(...) {
9     // std
10 }
11
12 int main(int argc, char* argv[]) {
13     /*
14      * argv[1]: 输入
15      * argv[2]: 选手输出
16      * exit code: 返回判断结果
17      */
18     FILE* fin = fopen(argv[1], "r");
19     FILE* fout = fopen(argv[2], "r");
20
21     double pans, jans;
22     fscanf(fout, "%lf", &pans);
23
24     jans = solve(...);
25     if (abs(pans - jans) < 1e-3)
26         return AC;
27     else
28         return WA;
29 }
```

SYZOJ 2

▼ Note

SYZOJ 2 有现成的修改版 [Testlib](#)，建议使用 Testlib。

LibreOJ 的最新版本已不再基于 SYZOJ，而是基于 [Lyrio](#)。Lyrio 支持使用原版 Testlib 编写评测器，这也是更加通用且推荐的做法。

```

1 #include <LTcmath>
2 #include <LTcstdio>
3
4 int main(int argc, char* argv[]) {
5     /*
6      * in: 输入
7      * user_out: 选手输出
8      * answer: 标准输出
9      * code: 选手代码
10     * stdout: 输出最终得分 (0 ~ 100)
11     * stderr: 输出错误报告
12     */
13     FILE* fin = fopen("input", "r");
14     FILE* fout = fopen("user_out", "r");
15     FILE* fstd = fopen("answer", "r");
16     FILE* fcode = fopen("code", "r");
17
18     double pans, jans;
19     fscanf(fout, "%lf", &pans);
20     fscanf(fstd, "%lf", &jans);
21
22     if (abs(pans - jans) < 1e-3) {
23         printf("%d", 100);
24         fprintf(stderr, "Good job\n");
25     } else {
26         printf("%d", 0);
27         fprintf(stderr, "Too big or too small, expected %f, found %f\n", jans,
28             pans);
29     }
30 }

```

牛客网

▼ Note

牛客网有现成的修改版 [Testlib](#)，建议使用 Testlib。

参见：[如何在牛客网出 Special Judge 的编程题](#)

```

1 #include <LTcmath>
2 #include <LTcstdio>
3
4 #define AC 0
5 #define WA 1
6
7 int main(int argc, char* argv[]) {
8     /*
9      * input: 输入
10     * user_output: 选手输出
11     * output: 标准输出
12     * exit code: 返回判断结果
13     */
14     FILE* fin = fopen("input", "r");
15     FILE* fout = fopen("user_output", "r");
16     FILE* fstd = fopen("output", "r");
17
18     double pans, jans;
19     fscanf(fout, "%lf", &pans);
20     fscanf(fstd, "%lf", &jans);
21
22     if (abs(pans - jans) < 1e-3)
23         return AC;
24     else
25         return WA;
26 }

```

DOMJudge

▼ Note

DOMJudge 支持任何语言编写的 spj, 参见: [problemarchive.org output validator 格式](https://problemarchive.org/output-validator-format)。

DOMJudge 有现成的修改版 [Testlib](#), 建议使用 Testlib。

DOMJudge 使用的 Testlib 及导入 Polygon 题目包方式的文档: <https://github.com/cn-xcpc-tools/testlib-for-domjudge>

DOMJudge 的 [默认比较器](#) 自带了浮点数带精度比较, 只需要在题目配置的 `validator_flags` 中添加 `float_tolerance 1e-3` 即可。

```

1 #include <LTCmath>
2 #include <LTCstdio>
3
4 #define AC 42
5 #define WA 43
6 char reportfile[50];
7
8 int main(int argc, char* argv[]) {
9     /*
10      * argv[1]: 输入
11      * argv[2]: 标准输出
12      * argv[3]: 评测信息输出的文件夹
13      * stdin: 选手输出
14      */
15     FILE* fin = fopen(argv[1], "r");
16     FILE* fstd = fopen(argv[2], "r");
17     sprintf(reportfile, "%s/judgemessage.txt", argv[3]);
18     FILE* freport = fopen(reportfile, "w");
19
20     double pans, jans;
21     scanf("%lf", &pans);
22     fscanf(fstd, "%lf", &jans);
23
24     if (abs(pans - jans) < 1e-3) {
25         fprintf(freport, "Good job\n");
26         return AC;
27     } else {
28         fprintf(freport, "Too big or too small, expected %f, found %f\n", jans,
29             pans);
30         return WA;
31     }
32 }

```

也可以使用 Kattis Problem Tools 提供的头文件 [validate.h](#) 编写，以实现更加复杂的功能。

参考资料

-
1. [LibreOJ 支持 testlib 检查器啦！](#) ↩
-

本页面最近更新：2024/8/13 21:15:50, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [Chrogeek](#), [Menci](#), [NachtgeistW](#), [sshwy](#), [Xeonacid](#), [yzy-1](#), [2014CAIS01](#), [CCXXLI](#), [countercurrent-time](#), [cuberctl](#), [Enter-tainer](#), [GitPinkRabbit](#), [HeRaNO](#), [SukkaW](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用